Notifications, Delivery History and Receipts

Design of Sapuni Madhushika's services: event-driven notification dispatch (email and mock SMS) via pluggable channels, customer delivery history and OpenPDF receipt generation in parcel-service, in-app notification centre with unread badge, and the complete LGTM observability stack.

Services notification-service (NestJS 11) · parcel-service (Spring Boot 4, receipt endpoint) **Data** MongoDB notifications · PostgreSQL parcels_db · Kafka

Branch feat/m9-notifications

1 Use cases

2 Data model

3 Architecture

4 Integration design

5 Key flows

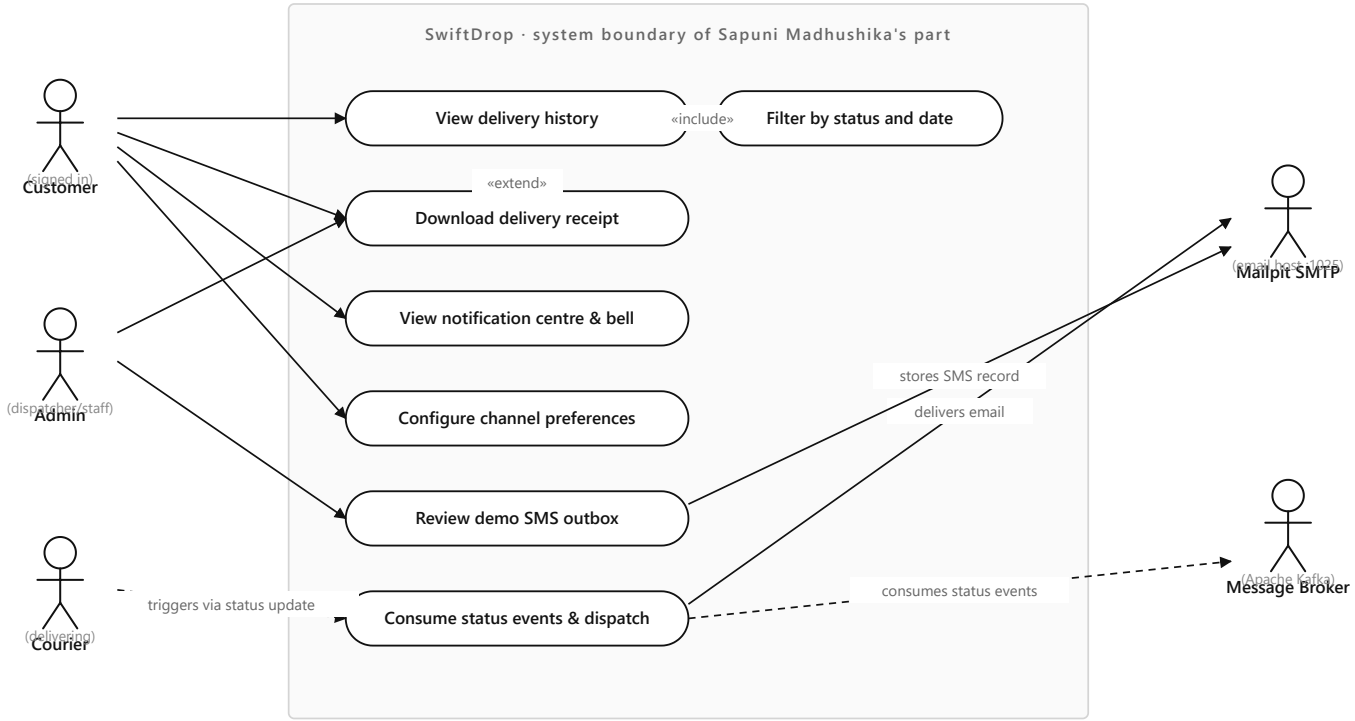
6 Design patterns and principles

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

1 Use cases

Sapuni Madhushika owns three capabilities. **Notification service** provides reliable, multi-channel customer communications (email via SMTP/Mailpit and mock SMS) triggered by status-change events. **Delivery history and receipts** let customers browse their historical orders and download cryptographically sound PDF delivery receipts. **Observability** gives the engineering team complete visibility across the platform.

FIGURE 1 **Use cases: notifications, delivery history and receipts**



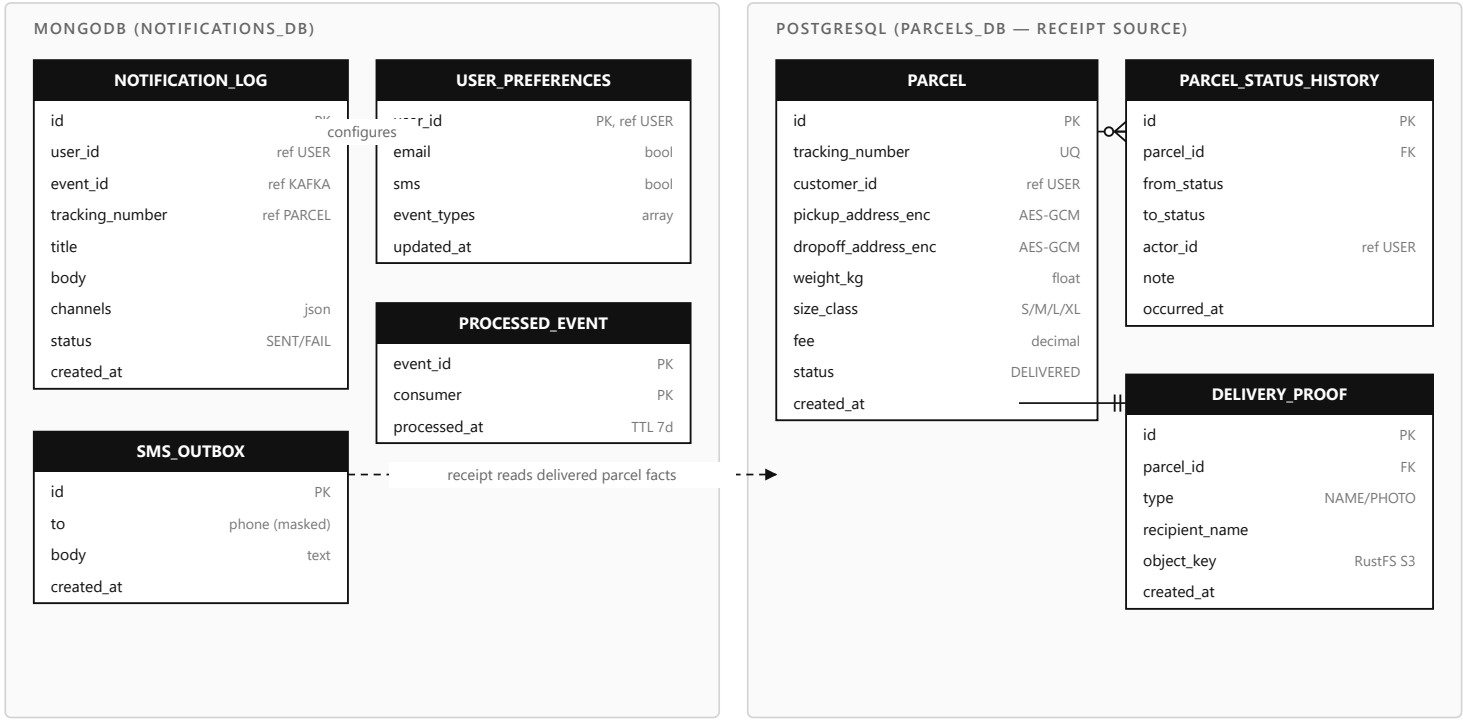
Customers view delivery history, download receipts, and configure preferences; couriers trigger notifications through status updates; admins review demo SMS records. «include» points to the use case that is always included; «extend» points to the use case it extends.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

2 Data model

Notifications are stored in MongoDB: notification-service maintains `notification_log` (in-app history), `user_preferences`, and `sms_outbox`. PROCESSED_EVENT makes event handling idempotent across Kafka consumers. Receipt generation reads delivered parcel facts directly from PostgreSQL (`parcels_db`) without cross-network joins.

FIGURE 2 **Data model: notifications MongoDB and receipt relational source**



notifications db holds document collections in MongoDB with a 7-day TTL on processed events. Receipts are projected in memory from parcel, status history, and delivery proof in parcels_db. Solid line with crow's foot: a real one-to-many relationship. Dashed arrow: records referenced by ID or derived by projection.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

3 Architecture

Proposal: microservices behind an API gateway, with the Next.js app acting as backend-for-frontend (BFF). Each service owns its data; services integrate through Kafka events or through the gateway.

Why this style for Sapuni Madhushika's services

- **Independent scaling:** notification fan-out and history browsing have completely different scaling profiles from write-heavy parcel booking.
- **Pluggable channels:** NestJS handles asynchronous I/O and template compilation efficiently, while Java OpenPDF ensures enterprise-grade PDF rendering in parcel-service.
- **Loose coupling:** notification-service subscribes to events, so order producers never know notification logic exists and notification downtime never blocks order booking.
- **Security by construction:** receipt downloads enforce strict tenant isolation via domain AccessPolicy (404 for foreign parcels; 409 for non-delivered parcels).

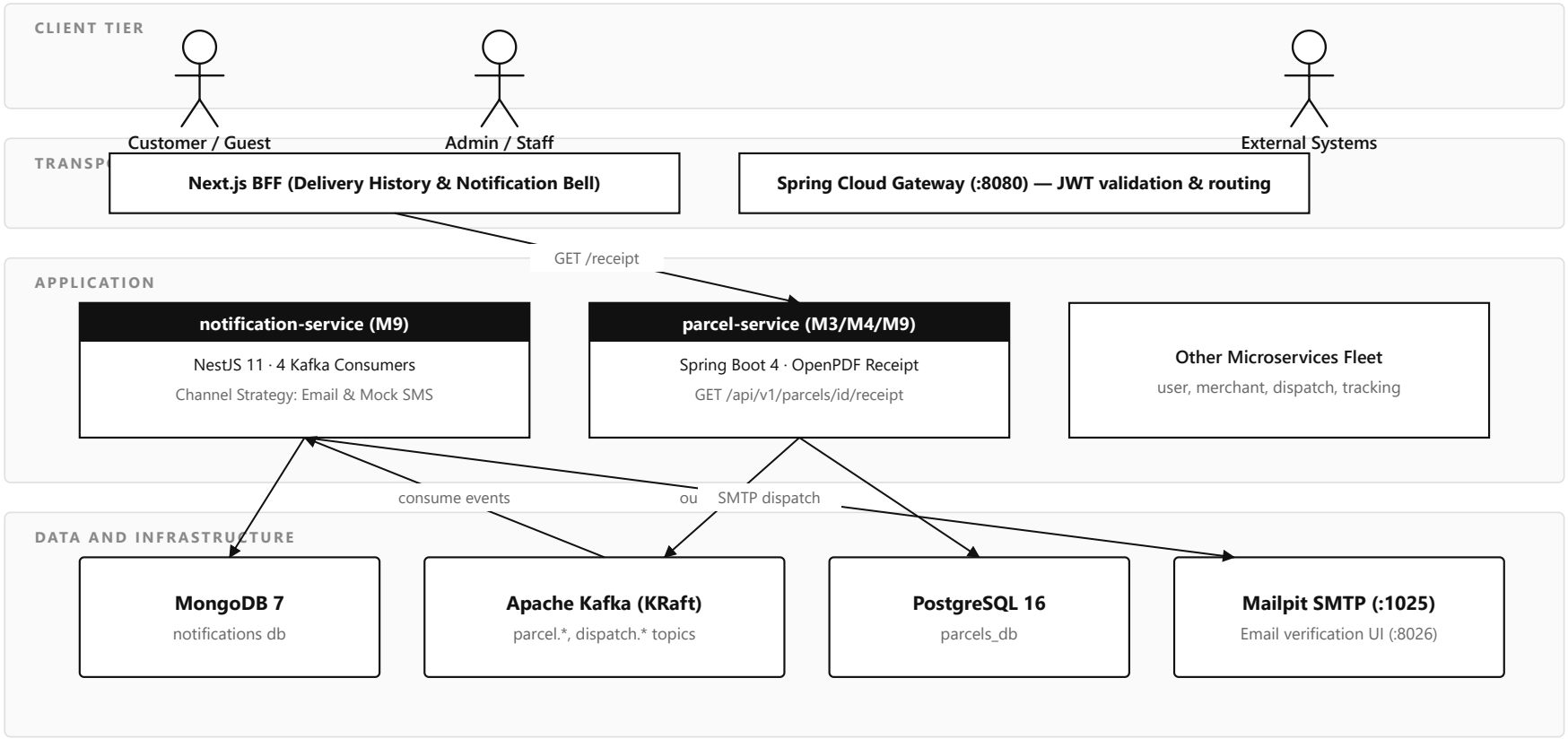
Trade-off accepted: notification delivery is eventually consistent (typically arriving within 1 to 2 seconds following status transitions).

Tiers

TIER	RESPONSIBILITIES
Client	Next.js delivery history page, notification bell, preferences form, admin demo SMS inbox
Transport	Next.js BFF (token stays server-side) and Spring Cloud Gateway: routing, JWT check, rate limits
Application	notification-service (consumers → channel strategy → templates), parcel-service (receipts)
Data	MongoDB `notifications`, PostgreSQL `parcels_db`, Kafka status topics
Platform	Full LGTM observability stack: Alloy collector, Loki logs, Tempo traces, Prometheus metrics, Grafana

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

FIGURE 3 **Architecture: layered microservices with a BFF**

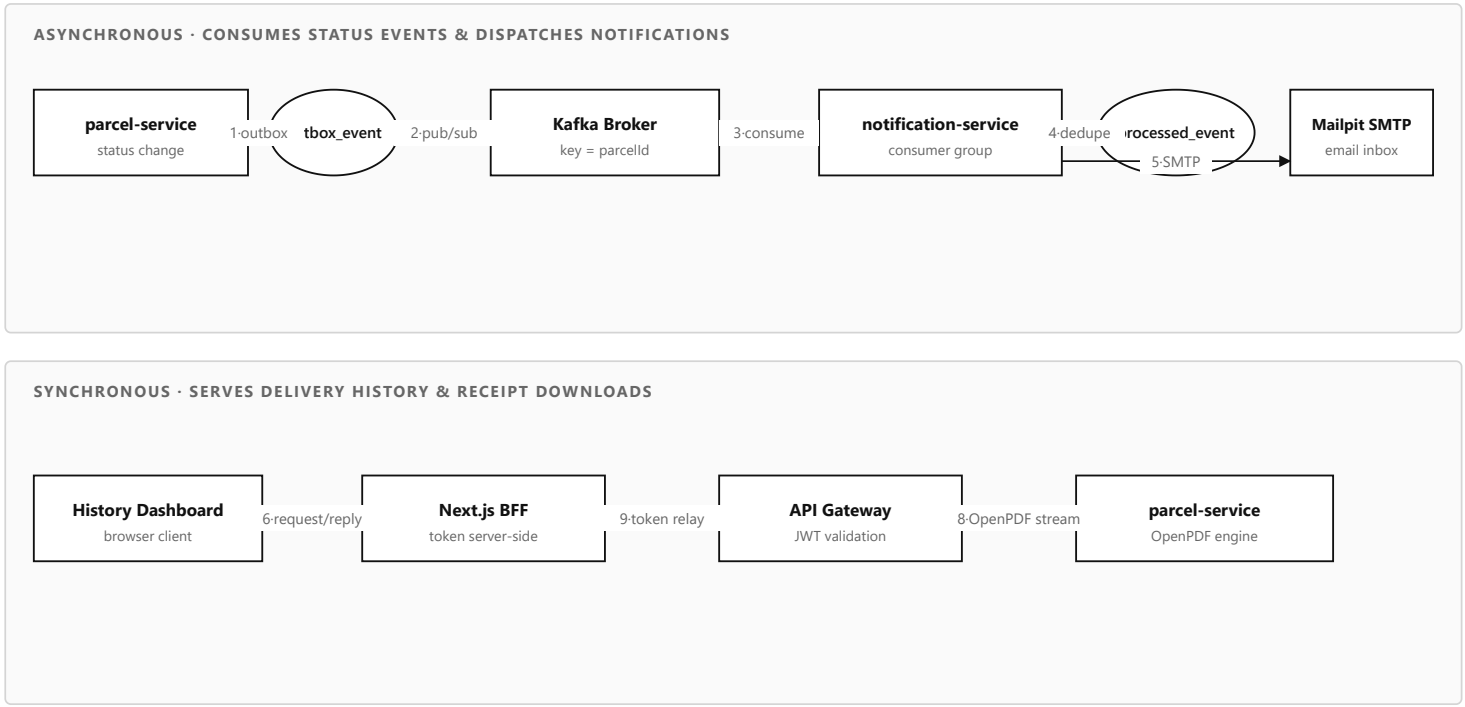


Four tiers with unified observability. Sapuni Madhushika's notification-service sits in the application tier consuming Kafka events, while receipt generation is embedded in parcel-service; all services export telemetry to Grafana Alloy.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

4 Integration design

FIGURE 4 **Integration design: the patterns Sapuni Madhushika's services rely on**



Numbers match the pattern table. Asynchronous patterns (1-5) ingest events and dispatch notifications; synchronous ones (6-10) serve users browsing history and downloading receipts.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

4 Integration design (continued)

Integration patterns applied across asynchronous event consumption and synchronous receipt generation:

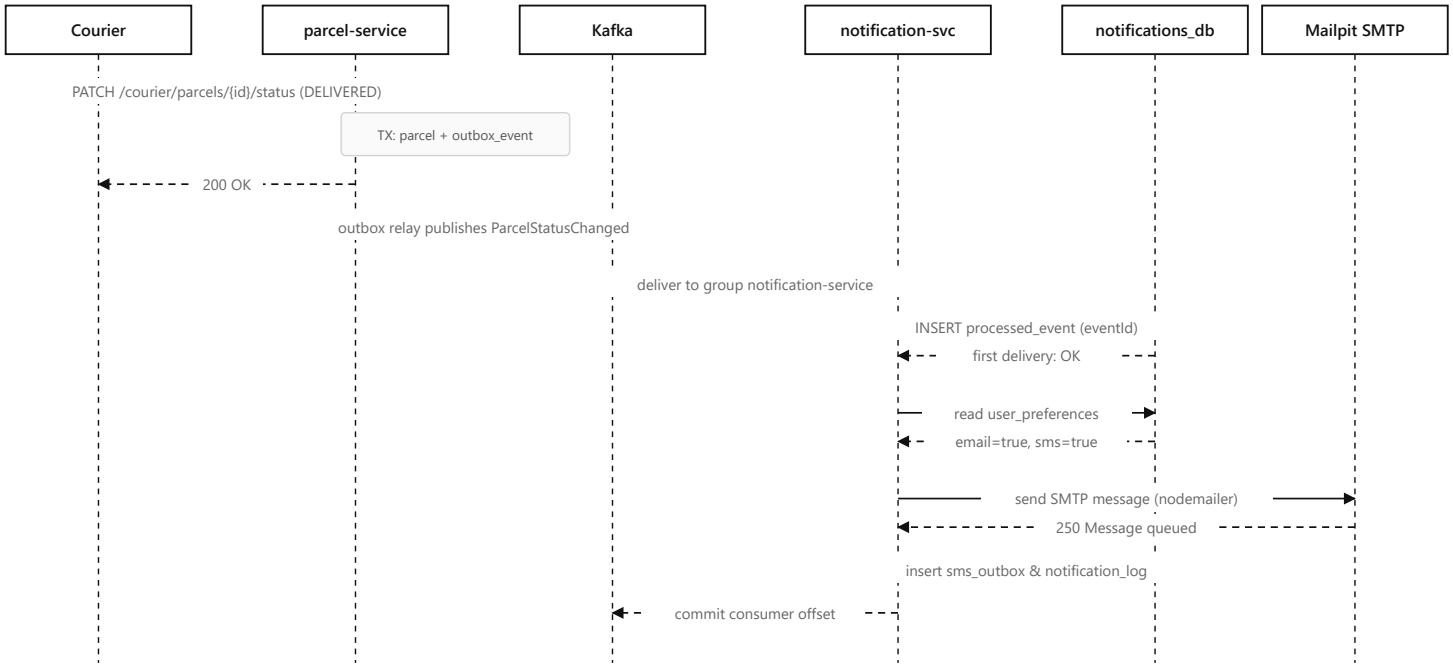
#	PATTERN	WHERE USED	WHY
1	**Transactional outbox**	`parcel-service` outbox_event table	Event and state change commit together, so notifications never miss a real status transition.
2	**Publish/subscribe (broker)**	Kafka topic `swiftdrop.parcel.status-changed.v1`	One status update feeds notifications, tracking, webhooks, and dispatch without producers knowing any of them.
3	**Competing consumers**	Consumer group `notification-service`	Replicas share Kafka partitions; key = parcelId keeps events in order per shipment.
4	**Idempotent consumer**	MongoDB `processed_event` collection	Kafka delivers at least once; duplicate deliveries are detected and rejected before sending duplicate emails.
5	**Strategy channel dispatch**	`NotificationChannel` (Email, Mock SMS)	Allows adding new delivery channels (e.g. real SMS provider) without altering consumer orchestration.
6	**Request/reply (REST)**	`/api/v1/parcels/{id}/receipt` via gateway	The browser needs an immediate binary PDF stream when a customer requests a proof receipt.
7	**Streaming response**	Spring MVC `ResponseEntity<Resource>`	Receipt PDF is generated and streamed in chunks without loading large binary files entirely in memory.
8	**Backend-For-Frontend (BFF)**	Next.js 16 App Router route handlers	Access tokens remain server-side in encrypted session cookies; client JS never touches bearer tokens.
9	**Gateway + token relay**	Spring Cloud Gateway to parcel-service	Validates Keycloak JWT signature once at the edge, relaying correlation headers and claims.
10	**Dead-letter queue (DLQ)**	Topic `swiftdrop.notification.dead-letter.v1`	Failed dispatches after 3 exponential backoff retries are preserved for administrative diagnosis.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

5 Key flows

Flow 5 is Sapuni Madhushika's part of the shared courier status update flow. Flows 6 to 10 cover receipt download, preference handling, live notification bell updates, admin SMS inspection, and channel dispatch.

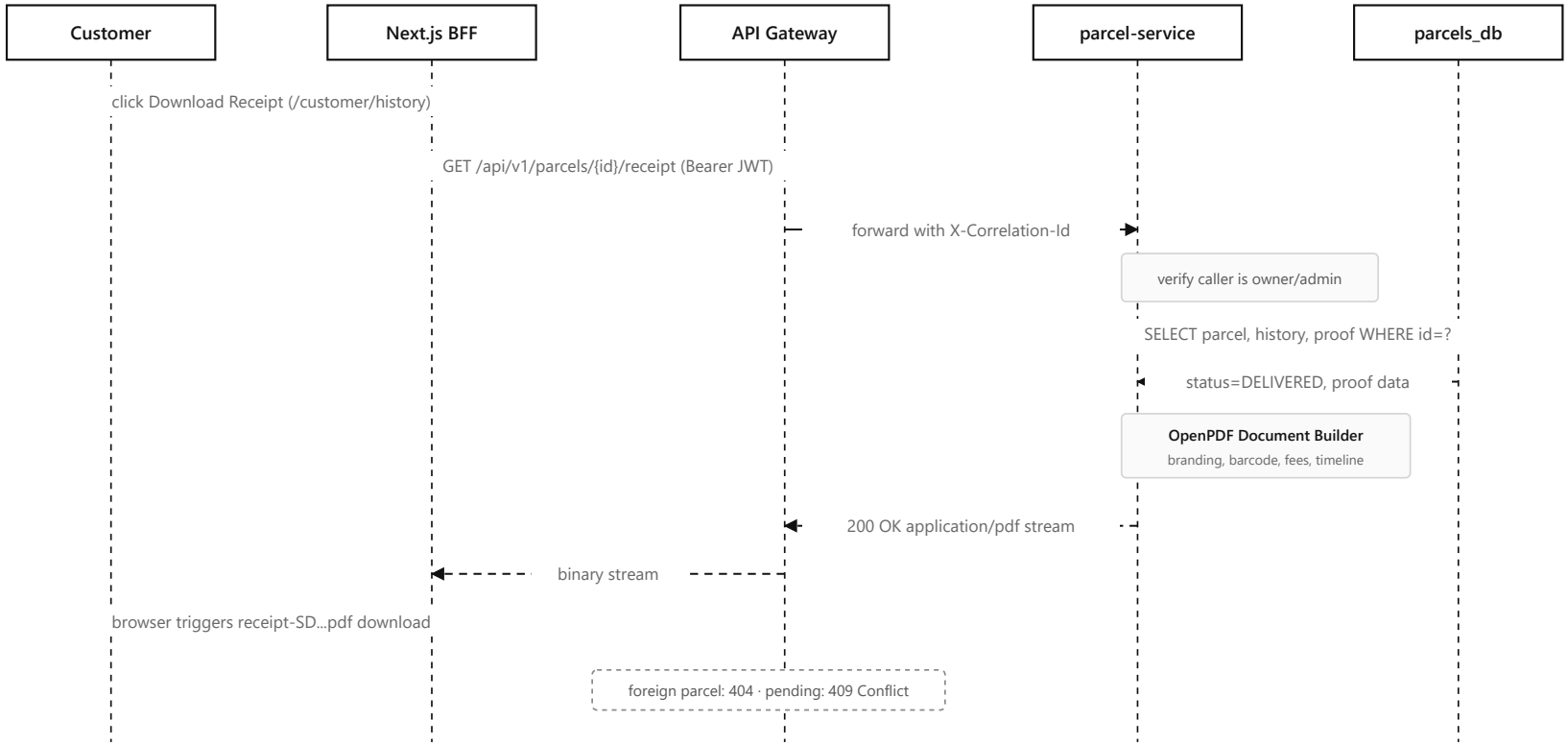
FIGURE 5 **Sequence: a courier status update triggers email and SMS notifications**



The event that triggers live tracking and webhooks is consumed independently by notification-service. A duplicate delivery is detected via processed_event and rejected without producing duplicate emails.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

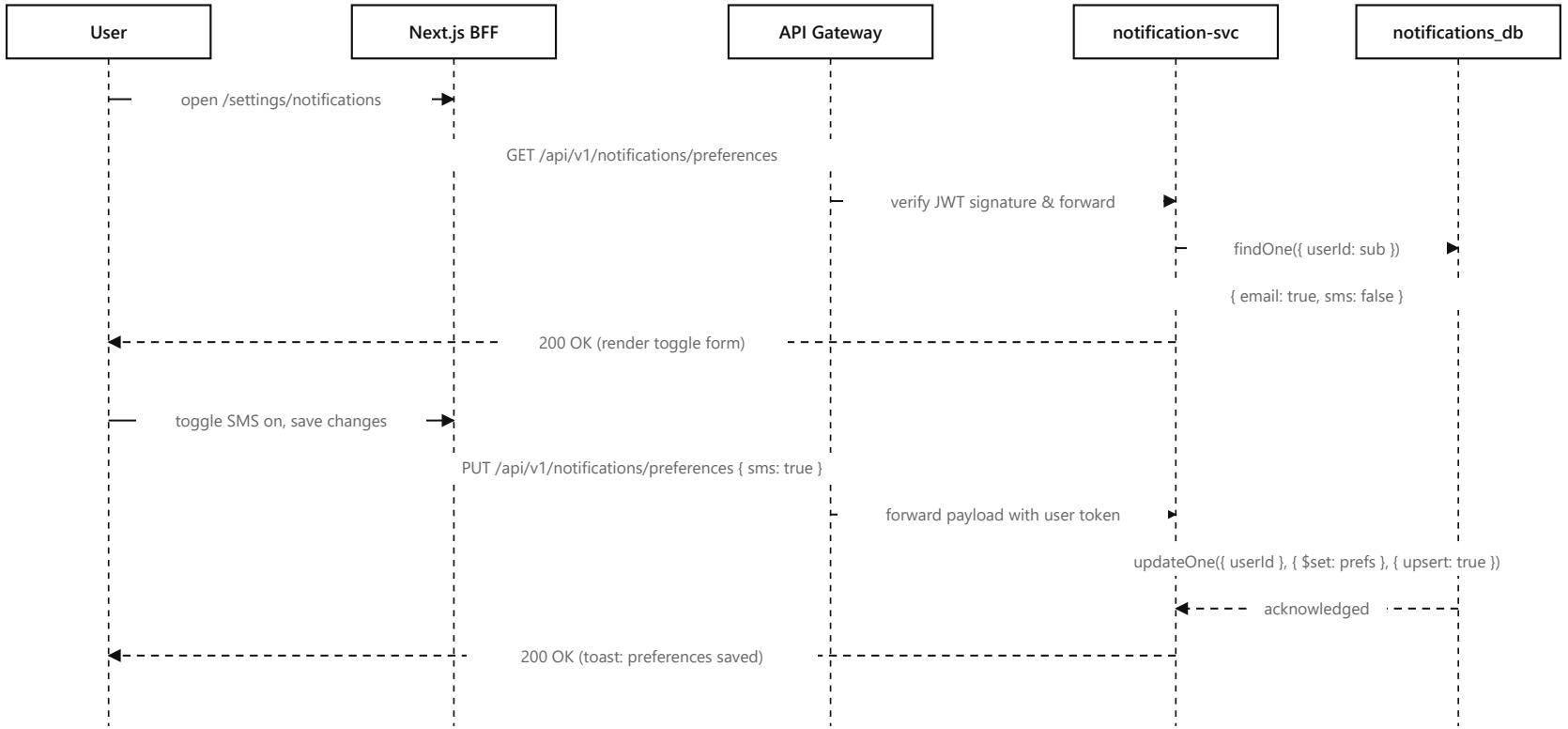
FIGURE 6 **Sequence: a customer downloads a delivery receipt (PDF)**



The browser never holds the access token: the BFF adds it server-side. ParcelAccessPolicy ensures customers cannot download receipts for other customers' parcels; non-delivered parcels return 409 Conflict.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

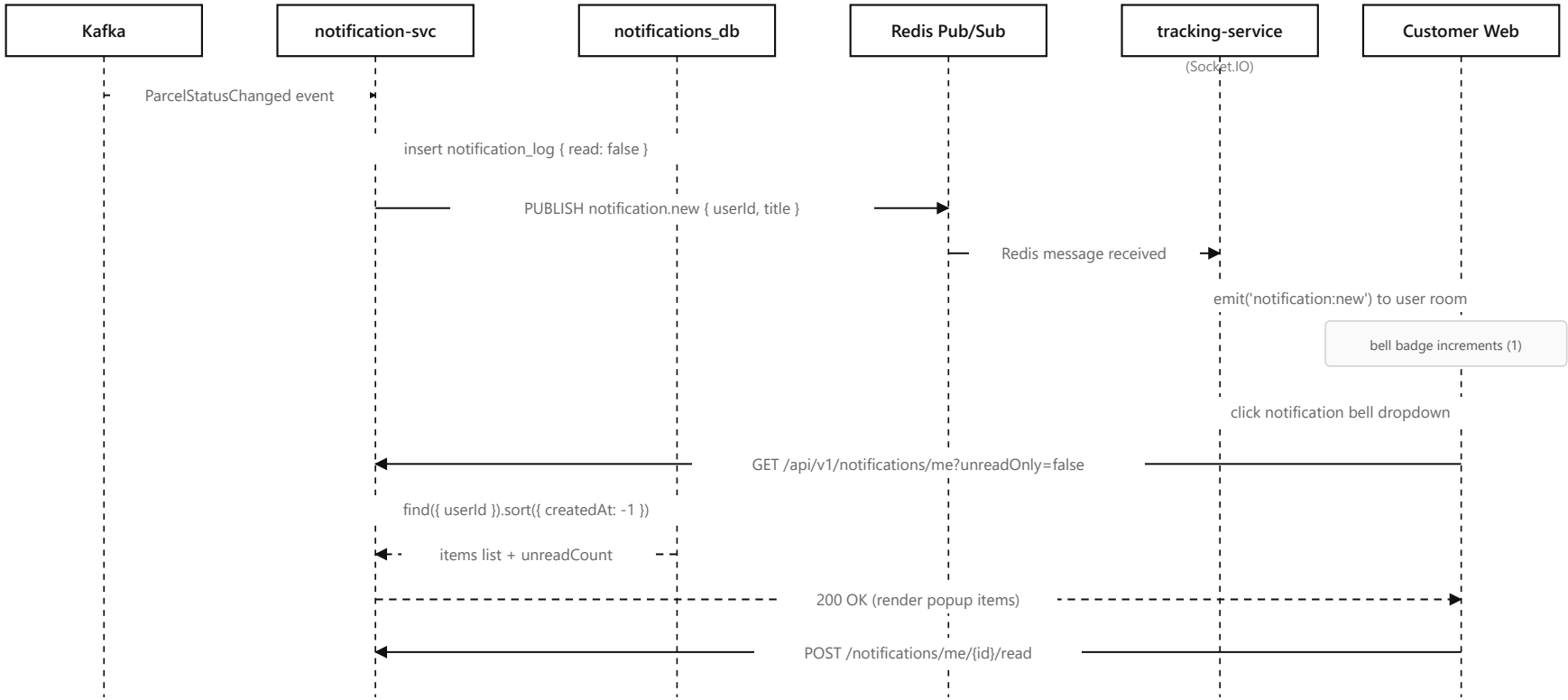
FIGURE 7 **Sequence: a user configures notification preferences**



User preferences are queried and updated through the Next.js BFF. The notification-service updates MongoDB with upsert semantics, returning an immediate acknowledgment.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

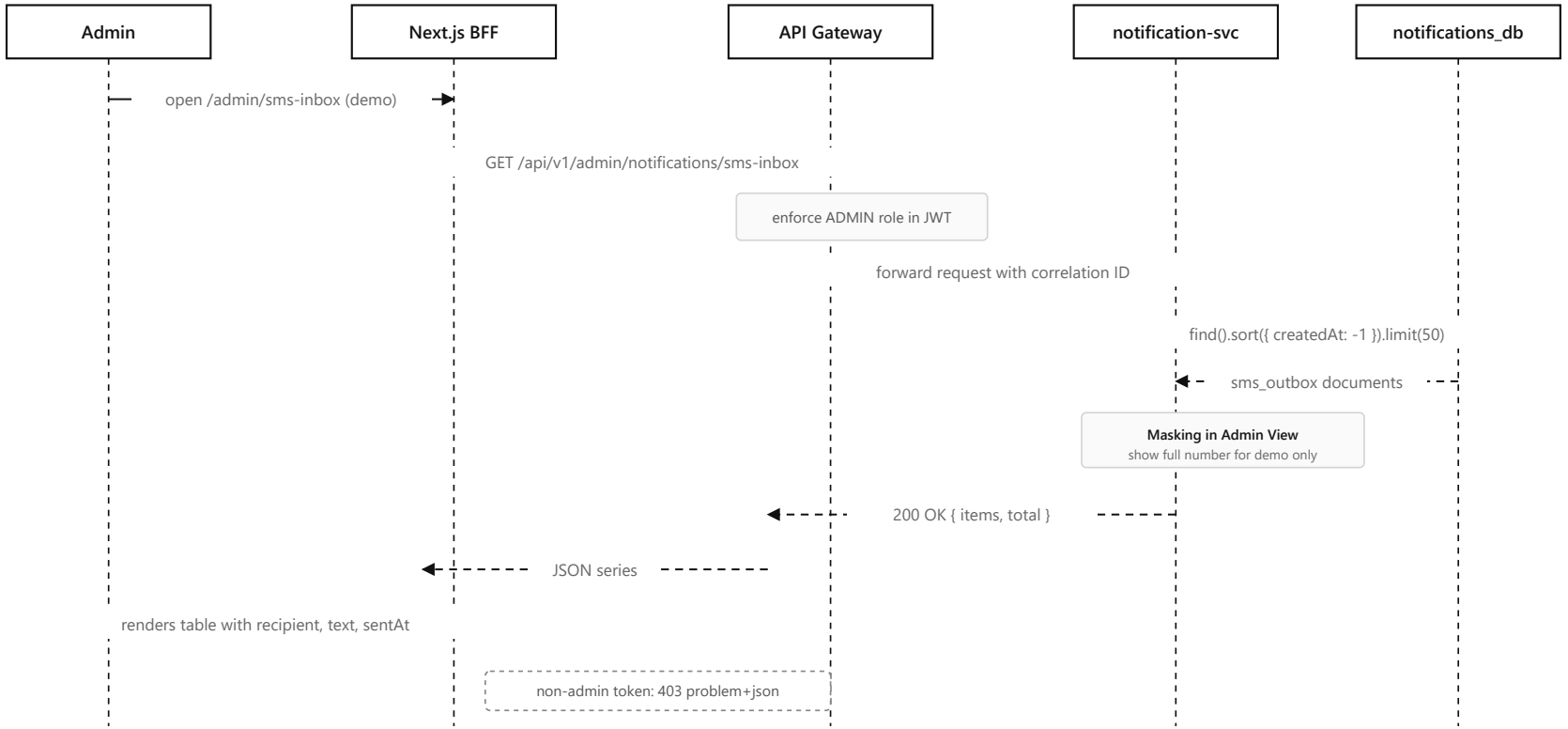
FIGURE 8 **Sequence: in-app notification & live bell update**



In addition to SMTP email and SMS outbox records, in-app notifications are persisted to MongoDB and broadcast via Redis pub/sub to tracking-service, updating the browser notification bell live.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

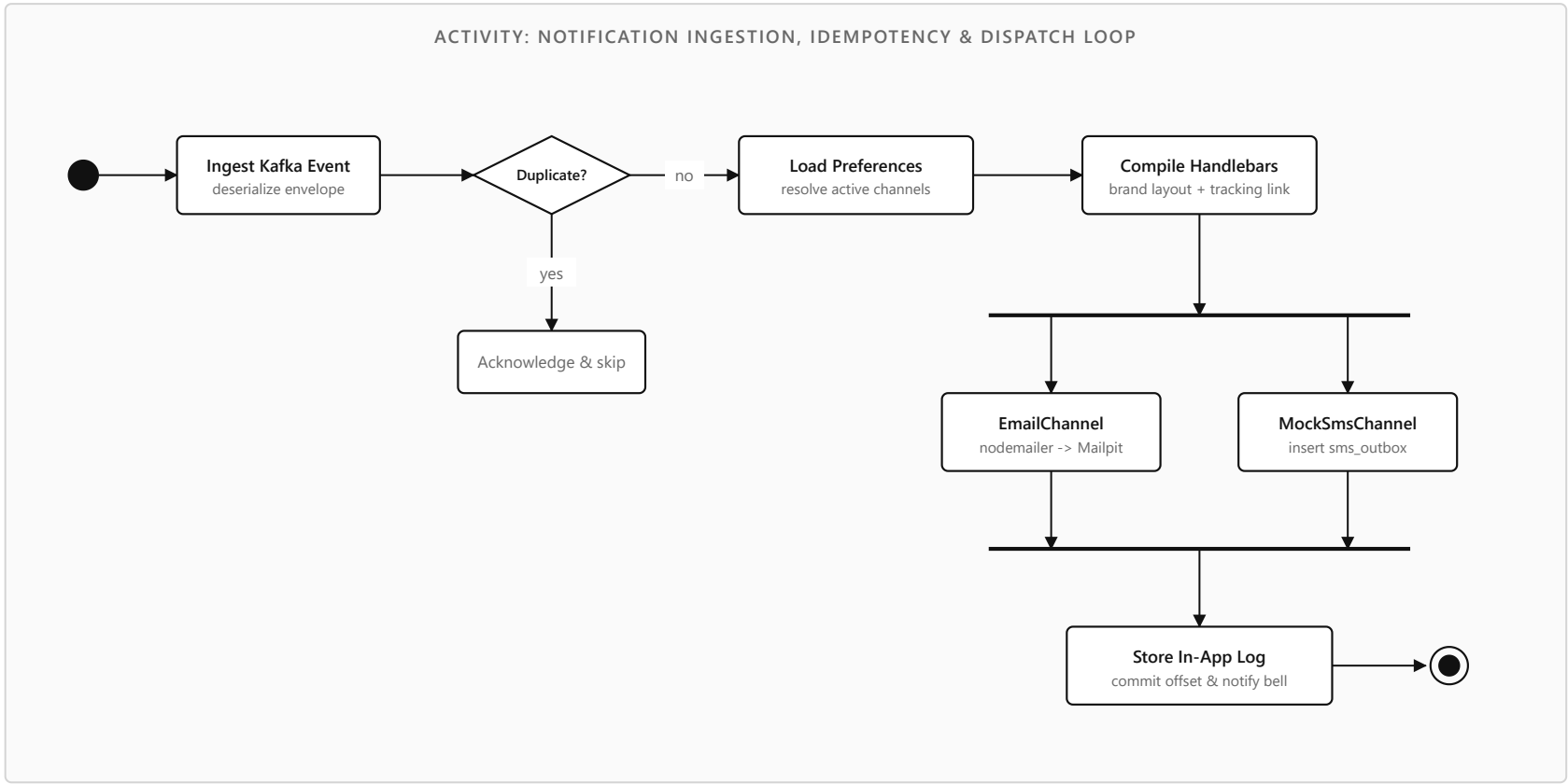
FIGURE 9 **Sequence: admin reviews the demo SMS inbox**



The demo SMS inbox gives evaluators and administrators instant visual confirmation of outgoing SMS dispatches. Gateway enforces the ADMIN role; non-admin users receive 403 problem+json.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

FIGURE 10 **Activity: notification dispatch and channel selection loop**

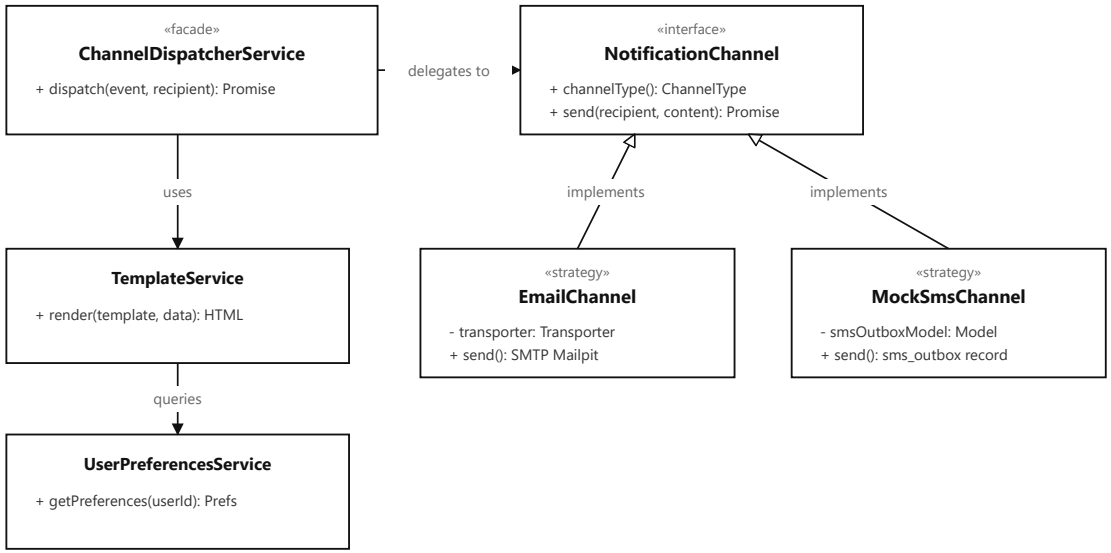


Every guardrail is a decision in this loop: deduplication check, preference resolution, parallel channel dispatch, audit logging, and dead-letter queue routing on persistent failure.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

6 Design patterns and principles

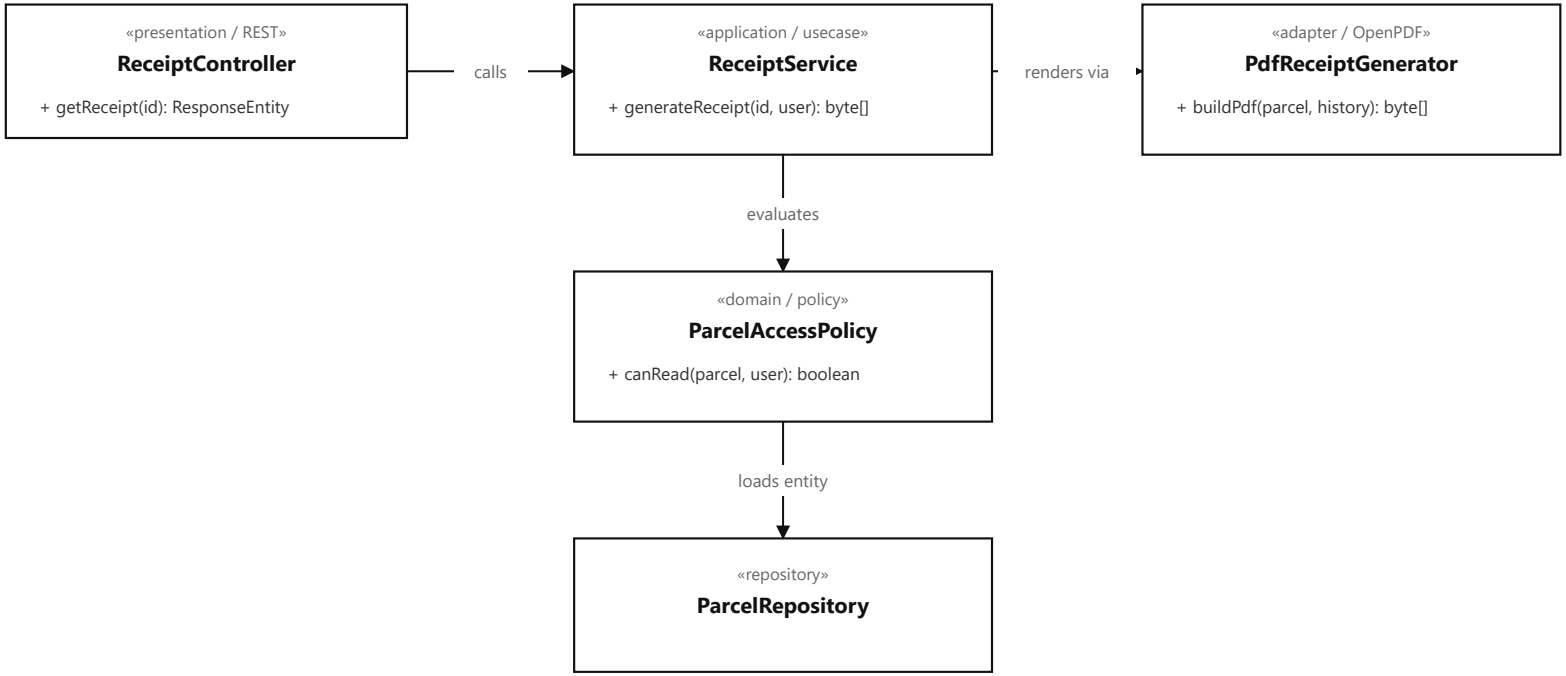
FIGURE 11 **Design patterns in notification-service**



Strategy for notification channels (Email, Mock SMS, future providers), Facade for channel dispatch, Adapter for SMTP, and Repository for MongoDB access. Dashed open arrow: depends on. Solid open arrow: composition. Dashed hollow triangle: implements.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

FIGURE 12 **Design patterns in parcel-service receipt generation**



Layered and hexagonal: controller and OpenPDF generator are adapters around domain policies and application services; access policy ensures strict tenant isolation.

Diagram sources: doc/diagrams/m9/*.svg in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

6 Design patterns and principles (continued)

Summary of design patterns and engineering principles applied across Sapuni Madhushika's vertical:

PATTERN	APPLIED TO
Strategy	`NotificationChannel`: EmailChannel and MockSmsChannel are interchangeable strategies.
Template Method	Shared consumer pipeline in `@swiftdrop/common`: dedupe, handle, commit.
Adapter	`EmailChannel` wraps nodemailer SMTP; `PdfReceiptGenerator` wraps OpenPDF document rendering.
Facade	`ChannelDispatcherService` hides preference querying, template compilation, and transport invocation.
Observer / Pub-Sub	`notification-service` reacts to Kafka parcel events it subscribes to.
Repository	Mongoose models for MongoDB collections; Spring Data repositories for parcel facts.
CQRS and Idempotent Consumer	Transactional writes commit in parcel-service; notification-service reads events and skips duplicate deliveries.

Principles

- **Single responsibility:** Consumers translate events; dispatcher coordinates channels; templates handle presentation; access policy validates security.
- **Open/closed:** A new notification channel (e.g., Twilio SMS or push notifications) is an added class implementing `NotificationChannel`, not an edit to consumers.
- **Dependency inversion:** Consumers and dispatcher depend on the `NotificationChannel` interface; persistence depends on repositories.
- **DRY:** Common layout headers and footers are encapsulated in Handlebars partials; Kafka consumer logic is shared via `@swiftdrop/common`.
- **Least privilege:** Receipt downloads verify caller ownership (customer can only access their own parcel); non-privileged calls return 404.
- **Twelve-factor configuration:** SMTP hosts, MongoDB URIs, and Kafka brokers are injected via environment variables and validated at startup with Zod.